

¿HAS ESCUCHADO ANTES DE LA ARQUITECTURA HEXAGONAL?

Este tipo de arquitectura para los softwares ha dado mucho de qué hablar últimamente, y no nos sorprende en absoluto.

Se trata de una arquitectura que te permite crear una aplicación capaz de funcionar sin una interfaz de usuario o base de datos. Así, podrás trabajar en ella aún cuando la base no esté disponible y conectar las aplicaciones sin la intervención del usuario.

¿QUÉ ES LA ARQUITECTURA HEXAGONAL?

La arquitectura hexagonal, a la que también se le conoce como **arquitectura de puertos y adaptadores**, es una arquitectura de software que se basa en la idea de aislar la lógica comercial central de las preocupaciones externas, por medio de la separación de la aplicación en componentes débilmente acoplados.

Pero, **¿por qué no lo ilustramos a través de un ejemplo?** Imaginemos que estamos creando una API que necesita obtener algunos datos de una [API REST](#).

En lugar de tener nuestra clase de caso de uso (o clase de servicio) **acoplada a la API REST externa**, podemos crear un puerto, que define la interfaz que nuestra clase de caso de uso tiene que implementar para obtener los datos.

Luego podemos crear un Adapter, que es una clase que implementa la interfaz Port y delega las llamadas a la **API REST externa u otro servicio externo**.

De esta forma, si necesitamos cambiar la manera en que obtenemos los datos, solo necesitamos crear **un nuevo adaptador que implemente la interfaz del puerto**, sin tener que cambiar la implementación del caso de uso.

Ahora bien, el hexágono es solo una forma elegante de **describir el núcleo de la aplicación** que se compone de objetos de dominio, casos de uso que operan en ellos y puertos de entrada y salida que proporcionan una interfaz con el mundo exterior.

LOS 3 PRINCIPIOS DE LA ARQUITECTURA HEXAGONAL

Este tipo de arquitectura de software se sustenta en 3 principios básicos a cumplir:

1. A LA IZQUIERDA, EL LADO DEL USUARIO

Este es el lado a través del cual el usuario o los programas externos interactuarán con la aplicación, y es la que contiene el código que permite dichas interacciones.

Por lo general, su código de interfaz de usuario, sus rutas HTTP para una API, **sus serializaciones JSON** para programas que consumen su aplicación están aquí.

Este es el lado donde encontramos a los actores que impulsan la Lógica de Negocios.

2. LA LÓGICA DE NEGOCIOS, EN EL CENTRO

Esta es la parte que queremos aislar de los lados izquierdo y derecho. Contiene todo el código que concierne e implementa la lógica empresarial.

El vocabulario empresarial y la lógica empresarial pura, que se relaciona con el problema concreto que resuelve su aplicación, todo lo que la hace rica y específica está en el centro.

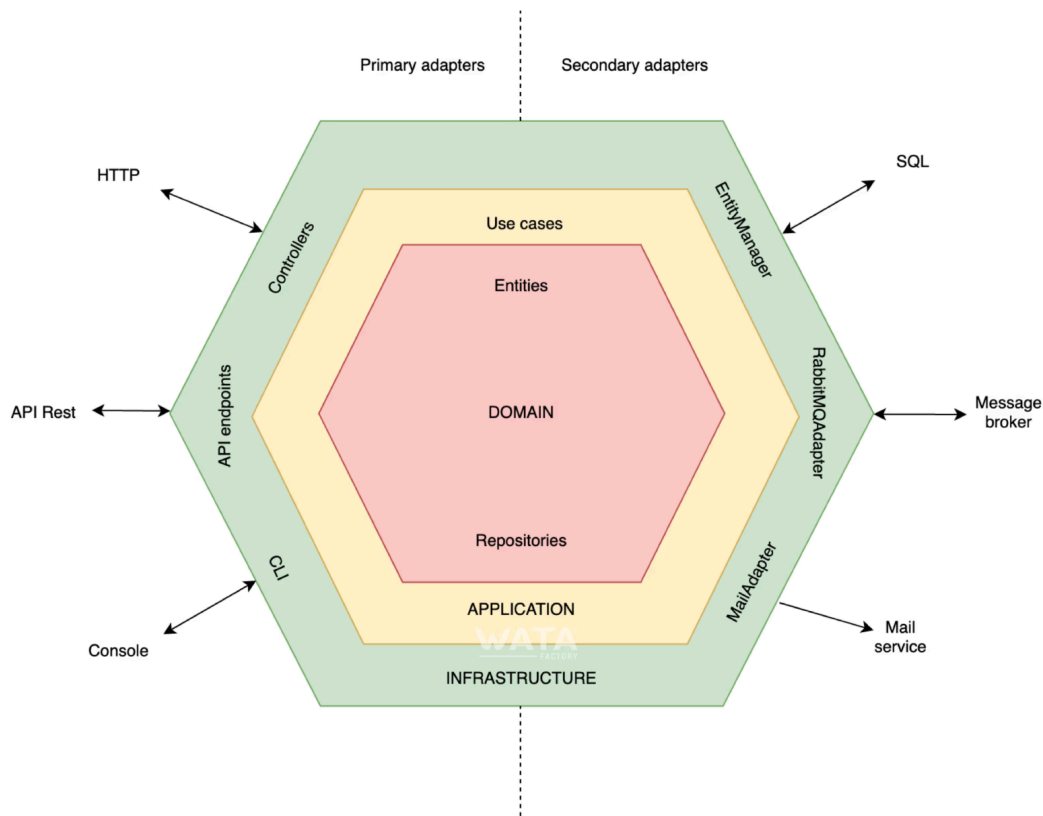
Idealmente, un experto en el dominio que no sabe cómo codificar, podría leer un fragmento de código en esta parte y señalarle una inconsistencia. De hecho, si lo piensas, **es algo que podría pasarte si eres el desarrollador**.

3. A LA DERECHA, EL LADO DEL SERVIDOR

Aquí es donde encontraremos lo que tu aplicación necesita, lo que hace que funcione.

Contiene detalles esenciales de la infraestructura, como **el código que interactúa con la base de datos**, realiza llamadas al sistema de archivos o el código que maneja las llamadas HTTP a otras aplicaciones de las que depende, por ejemplo.

Este es el lado donde encontramos a los actores que son manejados por la Lógica de Negocios.



¿POR QUÉ ES IMPORTANTE LA ARQUITECTURA HEXAGONAL?

Una primera característica importante de esta separación es que separa los problemas. En cualquier momento, puedes optar por centrarte en una única lógica, **casi de manera independiente de las otras dos**: lógica del lado del usuario, lógica empresarial o lógica del lado del servidor.

Son más fáciles de entender sin mezclarlos y las restricciones de cada lógica **tienen menos impacto en las demás**.

Otra característica es que ponemos la lógica empresarial al frente de nuestro código. Se puede aislar en un directorio o módulo para que sea explícito para todos los desarrolladores.

Puede definirse, refinarse y probarse sin asumir la carga cognitiva del resto del programa. Esto es importante porque al final, es la comprensión de los desarrolladores del negocio, es decir, lo que entra en producción.

Por último, en cuanto a las **pruebas que se realizan de manera automática** lograremos probar con un esfuerzo razonable:

Toda la lógica de negocios individualmente.

Integración entre el lado del usuario y la lógica de negocios independientemente del lado del servidor.

Integración entre la lógica de negocios y el lado del servidor de forma independiente en el lado del usuario

LOS ESTEREOTIPOS DE LA ARQUITECTURA HEXAGONAL

Como todo tipo de arquitectura para el desarrollo de software, encontramos algunos estereotipos o características que identifican a la arquitectura hexagonal, **¿te parece si vemos algunos de ellos en detalle?**

1. OBJETOS DE DOMINIO

En un dominio rico en reglas comerciales, los objetos de dominio son el elemento vital de una aplicación, y es que los objetos de dominio pueden contener tanto estado como comportamiento.

Cuanto más cerca esté el comportamiento del estado, más fácil será **entender, razonar y mantener el código**.

Los objetos de dominio no tienen ninguna dependencia externa. Son Java puro y proporcionan una API para casos de uso para operar en ellos.

Dado que los objetos de dominio **no dependen de otras capas de la aplicación**, los cambios en otras capas no los afectan. Pueden evolucionar libres de dependencias.

Este es un excelente ejemplo del principio de responsabilidad única, que establece que los componentes tienen que tener una sola razón para cambiar. **Para nuestro objeto de dominio**, este motivo es un cambio en los requisitos comerciales.

Tener una única responsabilidad nos permite **evolucionar nuestros objetos de dominio** sin tener que considerar dependencias externas. Esta capacidad de evolución hace que el estilo de arquitectura hexagonal sea perfecto para cuando se practica el diseño basado en dominios.

Durante el desarrollo, simplemente **seguimos el flujo natural de las dependencias**: comenzamos a codificar en los objetos del dominio y

salimos de allí. Si eso no es controlado por dominio, entonces ¿qué puede serlo?

2. CASOS DE USO

Conocemos los casos de uso como descripciones abstractas de lo que hacen los usuarios con nuestro software. En el estilo de arquitectura hexagonal, tiene sentido promover casos de uso a ciudadanos de primera clase de nuestra base de código.

Un caso de uso en este sentido es una clase que maneja todo, bueno, un cierto caso de uso. Como ejemplo, consideremos el caso de uso clásico: enviar dinero de una cuenta a otra en una aplicación bancaria.

Crearíamos una clase `SendMoneyUseCase` con una API distinta que permita a un usuario transferir dinero. **El código contiene todas las validaciones** y la lógica de las reglas comerciales que son específicas del caso de uso y, por lo tanto, no se pueden implementar dentro de los objetos del dominio.

Similar a los objetos de dominio, una clase de caso de uso no depende de los componentes externos. Cuando necesita algo de fuera del hexágono, **creamos un puerto de salida**.

3. PUERTOS DE ENTRADA Y SALIDA

Los objetos de dominio y los casos de uso están dentro del hexágono, es decir, dentro del núcleo de la aplicación. **Cada comunicación hacia y desde el exterior** ocurre a través de puertos dedicados.

Un puerto de entrada es una interfaz simple a la que pueden llamar componentes externos y que se implementa mediante un caso de uso. El componente que llama a dicho puerto de entrada se denomina **adaptador de entrada o adaptador de conducción**.

Un puerto de salida es, de nuevo, una interfaz simple a la que nuestros casos de uso pueden llamar si necesitan algo desde el exterior, como acceso a la base de datos, por ejemplo.

Esta interfaz está diseñada para adaptarse a las necesidades de los casos de uso, pero está implementada por un componente externo denominado **adaptador de salida o controlado**.

Si estás familiarizado con los principios sólidos, esta es una aplicación del principio de inversión de dependencia, porque estamos invirtiendo la dependencia de los casos de uso al adaptador de salida mediante una interfaz.

Con los puertos de entrada y salida en su lugar, tenemos lugares muy distintos donde los datos ingresan y salen de nuestro sistema, lo que facilita el razonamiento sobre la arquitectura.

4. ADAPTADORES

Los adaptadores forman la capa exterior de la arquitectura hexagonal, por lo que no son parte del núcleo pero interactúan con él.

Los adaptadores de entrada o adaptadores de conducción llaman a los puertos de entrada para hacer algo.

Un adaptador de entrada podría ser una interfaz web, por ejemplo. Cuando un usuario hace clic en un botón en un navegador, el adaptador web llama a un determinado puerto de entrada para llamar al caso de uso correspondiente.

Nuestros casos de uso llaman a los **adaptadores de salida o adaptadores controlados** y pueden, por ejemplo, proporcionar datos de una base de datos. Un adaptador de salida implementa un conjunto de interfaces de puerto de salida. Ten en cuenta que las interfaces están dictadas por los casos de uso y no al revés.

Los adaptadores facilitan el intercambio de una cierta capa de la aplicación. Si la aplicación tiene que poder utilizarse desde un cliente pesado además de la web, **agregamos un adaptador de entrada de cliente pesado**.

Si la aplicación necesita una base de datos diferente, agregamos un nuevo adaptador de persistencia que implementa las mismas interfaces de puerto de salida que el anterior.

VENTAJAS DE LA ARQUITECTURA HEXAGONAL

Para los desarrolladores, la arquitectura hexagonal ofrece varias ventajas, por lo que vale la pena mencionar algunas de ellas.

Retrasar las decisiones sobre tecnologías: la arquitectura hexagonal te permite decidir comenzar a escribir un proyecto y, de hecho, construirlo y probarlo, mientras pospone las decisiones tecnológicas que involucrarían conocimientos y datos que pueden no estar disponibles al comienzo de un proyecto.

Cambiar de tecnología con menor impacto: gracias a la arquitectura hexagonal, las empresas tecnológicas pueden decidir comenzar con la tecnología más simple (como la clásica base de datos relacional) y luego pasar a otra simplemente cambiando el adaptador.

Prueba la lógica empresarial aislada de los sistemas externos: Gracias a las interfaces, es posible crear stubs o simulacros para probar tu dominio central y tu lógica empresarial aislada de los sistemas externos. Esto confirmará que cualquier cambio en la tecnología no afectará la lógica comercial.

Remodele su dominio interno con menos esfuerzo en los sistemas externos: en cierto punto, el dominio interno evolucionará. Si el equipo técnico ha podido desacoplar con éxito y mantener en los adaptadores externos sólo el subconjunto de datos que necesitan, esto tendría que resultar en un precio más bajo y menos esfuerzo.

Sin embargo, no todo es bueno con la arquitectura hexagonal. Claro, que el único inconveniente es que este tipo de separación le costará más esfuerzo al equipo técnico al crear interfaces y mapear diferentes dominios.